Suffix Trie Construction

AlgoExpert · Medium · Tries

The problem

- Build a trie of all suffixes (terminated by '*') and answer contains(suffix).

Intuition

- Insert every suffix into a trie, then any substring is a path from the root.

Brute force vs optimal

- Store suffixes in a list → $O(n \cdot m)$ lookup
- Suffix trie → $O(n^2)$ build, $O(m)$ lookup
- Suffix tree/array → near-linear space

Approach

- 1. For each start index, insert the suffix
- 2. Mark ends with a star
- 3. contains walks the trie

Worked example

```
insert suffix 'babc*'

suffixes so far: ['babc']
```

Insert suffix 'babc'

Worked example

```
insert suffix 'abc*'

suffixes so far: ['babc', 'abc']
```

Insert suffix 'abc'

Worked example

```
insert suffix 'bc*'

suffixes so far: ['babc', 'abc', 'bc']
```

Insert suffix 'bc'

Worked example

```
insert suffix 'c*'

suffixes so far: ['babc', 'abc', 'bc', 'c']
```

Insert suffix 'c'

Worked example

```
contains('abc'): b? no — a → b → c → * ✓

contains('ab'): a → b, no * ✗
```

Lookup walks the trie, then checks the end marker

Java solution

```java
public class SuffixTrieConstruction {
    public static class TrieNode { public final Map<Character, TrieNode> children = new HashMap<>(); }

    public static class SuffixTrie {
        public final TrieNode root = new TrieNode();
        private static final char END = '*';

        public SuffixTrie(String s) {
            for (int i = 0; i < s.length(); i++) {
                TrieNode n = root;
                for (int j = i; j < s.length(); j++) n = n.children.computeIfAbsent(s.charAt(j), k -> new TrieNode());
                n.children.put(END, null);
            }
        }

        public boolean contains(String s) {
            TrieNode n = root;
            for (char c : s.toCharArray()) { n = n.children.get(c); if (n == null) return false; }
            return n.children.containsKey(END);
        }
    }
}
```

Complexity

- Time: build O(n^2), contains O(m)
- Space: O(n^2)

Edge cases & pitfalls

- End marker distinguishes suffix from substring
- Quadratic space
- Empty string

Interview tips & follow-ups

- Mention suffix trees / arrays

Recap

- Insert every suffix into a trie, then any substring is a path from the root.
- Complexity: build O(n^2), contains O(m)